MLWP Practical Lab codes with VIVA Questions

1. Problem Statement:

Load a dataset, perform data preprocessing, split the dataset into training and testing sets, train a simple classification model, and evaluate its performance. (Dataset: Iris.csv)

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report

# Step 1: Load dataset
df = pd.read_csv("Iris.csv")

# Step 2: Separate features (X) and target (y)
X = df.iloc[:, :-1]   # All columns except last
y = df.iloc[:, -1]    # Last column (species)

# Step 3: Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 4: Create and train the model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# Step 5: Make predictions
y_pred = model.predict(X_test)

# Step 6: Evaluate the model
accuracy = model.score(X_test, y_test)
print("Accuracy:", accuracy)

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

Most Frequent Viva Questions (with Answers)

1. What is the Iris dataset?

A dataset containing flower measurements used for classification into three species.

---

2. What are features and target?

- Features: input variables (X)
- Target: output variable (y)

3. Why do we split the dataset?

To train the model on one part and test it on unseen data.

---

4. What is KNN?

A classification algorithm that assigns a class based on nearest data points.

---

5. What does n_neighbors=3 mean?

The model looks at 3 nearest points to decide the class.

---

6. What is training data?

Data used to teach the model.

---

7. What is testing data?

Data used to evaluate the model.

---

8. What is accuracy?

Percentage of correct predictions.

---

9. What is precision?

Out of predicted positives, how many are correct.

---

10. What is recall?

Out of actual positives, how many are correctly predicted.

---

11. What is F1-score?

Combination of precision and recall.

---

12. What does model.fit() do?

Trains the model using training data.

---

13. What does model.predict() do?

Predicts output for new/unseen data.

---

14. What is classification?

Assigning data into categories.

---

15. Why use random_state?

To get the same data split every time for consistency.

---

16. Is KNN a supervised learning algorithm?

Yes, because it uses labeled data.

---

17. What happens if k is too small or too large?

- Small k → noisy results
- Large k → less accurate (over-generalization)

---

18. What type of algorithm is KNN?

Lazy learning algorithm (no training phase, stores data).

---

19. Why is classification_report used?

To get detailed evaluation metrics beyond accuracy.

---

20. What is overfitting?

Model performs well on training data but poorly on new data

---

2. **Load a dataset containing missing values, identify the missing data using appropriate technique, train a simple classification model, and evaluate its performance.**

(Dataset: Titanic.csv)

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Step 1: Load dataset
df = pd.read_csv("Titanic.csv")

# Step 2: Handle missing values
df['Age'].fillna(df['Age'].mean(), inplace=True)

# Step 3: Drop unnecessary columns
df.drop(['Cabin', 'Name', 'Ticket'], axis=1, inplace=True)

# Step 4: Encode categorical data
le = LabelEncoder()
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])

# Step 5: Separate features and target
X = df.drop("Survived", axis=1)
y = df["Survived"]

# Step 6: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 7: Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 8: Train model
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

# Step 9: Prediction
y_pred = model.predict(X_test)

# Step 10: Evaluation
accuracy = model.score(X_test, y_test)
print("Accuracy:", accuracy)

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

Most Frequent Viva Questions (with Answers)

1. What is the goal of this program?

To predict whether a Titanic passenger survived or not.

---

2. Why is missing data handled?

Because models cannot work with empty values.

---

3. Why use mean for Age?

It is a simple way to replace missing numeric values.

---

4. Why drop Cabin, Name, Ticket?

They are not useful for prediction or contain too many missing/irrelevant values.

---

5. What is LabelEncoder?

A tool to convert text data into numeric form.

---

6. Why convert categorical data?

Machine learning algorithms require numeric input.

---

7. What is feature scaling?

Adjusting values to a common scale.

---

8. Why is scaling important in KNN?

KNN uses distance, so larger values can dominate without scaling.

---

9. What is StandardScaler?

It standardizes data (mean = 0, standard deviation = 1).

---

10. What is KNN?

A classification algorithm based on nearest data points.

---

11. What does n_neighbors=5 mean?

Model considers 5 nearest points to make a decision.

---

12. What is train_test_split?

Splits data into training and testing sets.

---

13. What is accuracy?

Percentage of correct predictions.

---

14. What is precision?

Correct positive predictions out of total predicted positives.

---

15. What is recall?

Correct positive predictions out of actual positives.

---

16. What is F1-score?

Balance between precision and recall.

---

17. What does fit() do?

Trains the model.

---

18. What does predict() do?

Predicts output for new data.

---

19. What is supervised learning?

Learning using labeled data.

---

20. What happens if scaling is not done in KNN?

Model may give incorrect results due to uneven feature importance.

3. **Demonstrate unsupervised learning by applying clustering techniques on unlabeled data and interpreting the patterns formed.**

(Dataset: Mall_Customers.csv)

```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Step 1: Load dataset
df = pd.read_csv("Mall_Customers.csv")

# Step 2: Select features for clustering
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

# Step 3: Create and train KMeans model
model = KMeans(n_clusters=5, random_state=42)
model.fit(X)

# Step 4: Get cluster labels
labels = model.labels_

# Step 5: Visualize clusters
plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=labels)
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Customer Segments")
plt.show()
```

Most Frequent Viva Questions (with Answers)

1. What is the objective of this program?

To group customers into different segments based on income and spending.

---

2. What is clustering?

Grouping similar data points together.

---

3. What type of learning is KMeans?

Unsupervised learning.

---

4. Why only two features are selected?

To simplify clustering and visualization.

5. What is n_clusters?

Number of groups to divide the data into.

---

6. What does KMeans do?

It groups data based on similarity using distance.

---

7. What are cluster labels?

Numbers assigned to each data point indicating its group.

---

8. What is centroid?

Center point of a cluster.

---

9. How does KMeans work?

1. Select number of clusters
2. Assign points to nearest centroid
3. Update centroids
4. Repeat until stable

---

10. Why is random_state used?

To get the same result every time.

---

11. What is inertia in KMeans?

Sum of distances of points from their centroids.

---

12. What is the role of distance in KMeans?

Used to decide which cluster a point belongs to.

---

13. What is the output of KMeans?

Cluster labels and centroids.

---

14. What does fit() do here?

Trains the clustering model.

---

15. Why use scatter plot?

To visualize clusters clearly.

---

16. What do colors in the plot represent?

Different clusters.

---

17. What happens if we change n_clusters?

Number of groups will change.

---

18. Is KMeans sensitive to scale?

Yes, features should ideally be scaled.

---

19. What are real-world uses of KMeans?

Customer segmentation, image compression, pattern recognition.

---

20. What is the limitation of KMeans?

- Needs predefined number of clusters
- Sensitive to outliers
- Works best with spherical clusters

4. Demonstrate unsupervised learning by applying K-Means clustering on the Iris dataset. Use the Elbow Method to determine the optimal number of clusters and visualize the clusters formed.

```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
# Step 1: Load dataset
df = pd.read_csv("Iris.csv")

# Step 2: Select features (exclude target column)
X = df.iloc[:, :-1]

# Step 3: Use Elbow Method to find optimal number of clusters
wcss = []  # Within-Cluster Sum of Squares

for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

# Step 4: Plot Elbow Graph
plt.plot(range(1, 11), wcss)
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()

# Step 5: Train final model with optimal clusters (k=3)
model = KMeans(n_clusters=3, random_state=42)
model.fit(X)

# Step 6: Get cluster labels
labels = model.labels_

# Step 7: Visualize clusters (using first two features)
plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=labels)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Iris Clusters")
plt.show()
```

Most Frequent Viva Questions (with Answers)

1. What is the goal of this program?

To find clusters in Iris dataset using KMeans.

---

2. What is clustering?

Grouping similar data points together.

---

3. What is WCSS?

Sum of squared distances between points and their cluster centers.

---

4. Why calculate WCSS?

To evaluate clustering quality.

---

5. What is Elbow Method?

A method to find optimal number of clusters.

---

6. What is elbow point?

Point where WCSS stops decreasing rapidly.

---

7. Why is k=3 chosen?

Because elbow occurs at 3.

---

8. What is KMeans?

An algorithm that groups data based on similarity.

---

9. What is inertia_?

Same as WCSS in KMeans.

---

10. What does fit() do?

Trains the clustering model.

---

11. What are cluster labels?

Group numbers assigned to each data point.

---

12. Why ignore target column?

Clustering is unsupervised (no labels needed).

---

13. What is centroid?

Center of a cluster.

---

14. Why use scatter plot?

To visualize clusters.

---

15. What do colors represent?

Different clusters.

---

16. What happens if k is too large?

Over-clustering (too many small groups).

---

17. What happens if k is too small?

Under-clustering (less meaningful groups).

---

18. Is KMeans supervised?

No, it is unsupervised.

---

19. What is limitation of Elbow Method?

Sometimes elbow point is not clearly visible.

---

20. What is real-world use of this method?

Customer segmentation, grouping similar items, pattern recognition.

5. **Develop a machine learning model using Linear Regression to predict house prices based on various features such as area, number of rooms, location, and other relevant factors. Perform data preprocessing, train the model, and evaluate its performance.**

(Dataset: Housing.csv)

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Step 1: Load dataset
df = pd.read_csv("housing.csv")

# Step 2: Handle missing values
df.fillna(df.mean(numeric_only=True), inplace=True)

# Step 3: Separate features and target
X = df.drop("Price", axis=1)
y = df["Price"]

# Step 4: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 5: Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Step 6: Prediction
y_pred = model.predict(X_test)

# Step 7: Evaluation
r2 = model.score(X_test, y_test)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)

print("R2 Score:", r2)
print("MAE:", mae)
print("MSE:", mse)
```

Most Frequent Viva Questions (with Answers)

1. What is Linear Regression?

A method to predict continuous values using a linear relationship.

---

2. What is the goal of this program?

To predict house prices based on features.

---

3. What type of problem is this?

Regression problem.

---

4. What are features and target?

- Features: input variables
- Target: output (Price)

---

5. Why handle missing values?

Model cannot process empty data.

---

6. What does fit() do?

Trains the model.

---

7. What does predict() do?

Predicts output for new data.

---

8. What is R2 score?

Measure of how well model explains variance.

---

9. What does R2 = 1 mean?

Perfect prediction.

---

10. What is MAE?

Average absolute error.

---

11. What is MSE?

Average squared error.

---

12. Why use MSE?

To penalize large errors more.

---

13. What is regression?

Predicting continuous values.

---

14. What is difference between classification and regression?

- Classification → categories
- Regression → continuous values

---

15. What assumptions does Linear Regression make?

- Linear relationship
- No multicollinearity
- Constant variance

---

16. What happens if data is not linear?

Model accuracy decreases.

---

17. What is overfitting?

Model fits training data too well but fails on new data.

---

18. What is underfitting?

Model is too simple to capture patterns.

---

19. What is coefficient in Linear Regression?

Weight of each feature.

---

20. What is intercept?

Value when all inputs are zero.

6. Apply Principal Component Analysis (PCA) to reduce the dimensionality of the dataset and visualize the transformed data.

(Dataset: MNIST.csv)

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Load dataset
df = pd.read_csv("mnist.csv")

# Step 2: Separate features and target
X = df.drop("label", axis=1)
y = df["label"]

# Step 3: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Step 4: Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 5: Apply PCA
pca = PCA(n_components=50)
X_train = pca.fit_transform(X_train)
X_test = pca.transform(X_test)

# Step 6: Train model
model = LogisticRegression(max_iter=50, random_state=42)
model.fit(X_train, y_train)

# Step 7: Prediction
y_pred = model.predict(X_test)

# Step 8: Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Most Frequent Viva Questions (with Answers)

1. What is PCA?

A technique to reduce number of features while keeping important information.

---

2. Why use PCA?

To reduce dimensionality and improve performance.

---

3. What is dimensionality reduction?

Reducing number of input features.

4. What are principal components?

New features created by PCA that capture maximum variance.

---

5. What is variance in PCA?

Measure of data spread; PCA keeps features with high variance.

---

6. Why scale data before PCA?

Because PCA is affected by feature scale.

---

7. What does n_components=50 mean?

Reduce data to 50 features.

---

8. What is MNIST dataset?

Dataset of handwritten digits.

---

9. What is Logistic Regression?

A classification algorithm.

---

10. Why use Logistic Regression here?

To classify digits (multi-class problem).

---

11. What is stratify=y?

Ensures equal class distribution in train and test.

---

12. What is confusion matrix?

Table showing correct and incorrect predictions.

---

13. What is accuracy?

Percentage of correct predictions.

14. What is precision?

Correct positive predictions out of predicted positives.

---

15. What is recall?

Correct positive predictions out of actual positives.

---

16. What is F1-score?

Balance between precision and recall.

---

17. Does PCA lose information?

Yes, but tries to keep most important information.

---

18. What type of learning is PCA?

Unsupervised learning.

---

19. What is benefit of PCA?

Reduces computation and noise.

---

20. What is limitation of PCA?

Reduced interpretability of features.

7. **Perform feature selection using the correlation-based filter method and analyze the importance of features.**

(Dataset: Titanic.csv)

```
# Install seaborn (run once if not installed)
# pip install seaborn

# Import required libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Step 1: Load dataset
df = pd.read_csv("Titanic.csv")

# Step 2: Keep only numeric columns
df = df.select_dtypes(include='number')

# Step 3: Handle missing values
df.fillna(df.mean(numeric_only=True), inplace=True)

# Step 4: Compute correlation matrix
corr_matrix = df.corr()

# Step 5: Correlation with target variable
target_corr = corr_matrix["Survived"].sort_values(ascending=False)

print("Correlation with Target:\n")
print(target_corr)

# Step 6: Visualize correlation matrix
plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, cmap="coolwarm", annot=False)
plt.title("Correlation Matrix")
plt.show()
```

Most Frequent Viva Questions (with Answers)

1. What is feature selection?

Process of selecting important input features.

---

2. What is correlation?

Measure of relationship between two variables.

---

3. What is correlation range?

From -1 to +1.

---

4. What does +1 mean?

Strong positive relationship.

---

5. What does -1 mean?

Strong negative relationship.

---

6. What does 0 mean?

No relationship.

---

7. Why select only numeric columns?

Correlation works only on numeric data.

---

8. What is correlation matrix?

Table showing correlation between all features.

---

9. Why analyze correlation with target?

To find important features affecting output.

---

10. What is heatmap?

Graphical representation of correlation values.

---

11. What is multicollinearity?

When features are highly correlated with each other.

---

12. Why is multicollinearity a problem?

It can reduce model performance.

---

13. What is filter method?

Feature selection method independent of model.

---

14. Why use correlation for feature selection?

Simple and fast method.

---

15. What is advantage of feature selection?

Reduces complexity and improves accuracy.

---

16. What is disadvantage of correlation method?

Only captures linear relationships.

---

17. What is target variable here?

Survived.

---

18. What does high correlation with target mean?

Feature strongly affects output.

---

19. What is seaborn used for?

Data visualization.

---

20. Why visualize correlation matrix?

To easily understand relationships between features.

8. Implement standardization (Z-score normalization) and study its effect on model performance.

(Dataset: wine.csv)

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler

# Step 1: Load dataset
df = pd.read_csv("wine.csv")

# Step 2: Separate features and target
X = df.drop("Class", axis=1)
y = df["Class"]

# Step 3: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------
# WITHOUT STANDARDIZATION
```

```
# ----------------------------
model1 = LogisticRegression(max_iter=50, random_state=42)
model1.fit(X_train, y_train)

y_pred1 = model1.predict(X_test)
acc1 = accuracy_score(y_test, y_pred1)

# ----------------------------
# WITH STANDARDIZATION
# ----------------------------
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model2 = LogisticRegression(max_iter=50, random_state=42)
model2.fit(X_train_scaled, y_train)

y_pred2 = model2.predict(X_test_scaled)
acc2 = accuracy_score(y_test, y_pred2)

# Step 4: Compare results
print("Accuracy WITHOUT Scaling:", acc1)
print("Accuracy WITH Scaling:", acc2)
```

Most Frequent Viva Questions (with Answers)

1. What is standardization?

Process of scaling data to mean 0 and standard deviation 1.

---

2. What is Z-score normalization?

Another name for standardization.

---

3. Why is scaling needed?

To bring all features to same scale.

---

4. What happens without scaling?

Features with larger values dominate the model.

---

5. What is StandardScaler?

Tool to perform standardization.

6. Why fit only on training data?

To avoid data leakage.

---

7. What is data leakage?

Using test data information during training.

---

8. What is Logistic Regression?

A classification algorithm.

---

9. Why does scaling improve performance?

It helps model learn properly without bias.

---

10. What is accuracy?

Percentage of correct predictions.

---

11. What is mean in standardization?

Average value of feature.

---

12. What is standard deviation?

Measure of data spread.

---

13. What is formula of Z-score?

Z = (X - mean) / std

---

14. Which algorithms require scaling?

KNN, SVM, Logistic Regression, Neural Networks.

---

15. Which algorithms do not need scaling?

Decision Trees, Random Forest.

---

16. What is difference between normalization and standardization?

- Normalization → values between 0 and 1
- Standardization → mean 0, std 1

---

17. Why transform test data separately?

To use same scaling learned from training.

---

18. What is effect of scaling on gradient descent?

Faster convergence.

---

19. Can scaling reduce accuracy?

Sometimes, but usually improves it.

---

20. What is main advantage of standardization?

Better model performance and stability.

9. Apply Min-Max scaling on the dataset and compare the model performance before and after scaling.

(Dataset: wine.csv)

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Step 1: Load dataset
df = pd.read_csv("wine.csv")

# Step 2: Separate features and target
X = df.drop("Class", axis=1)
y = df["Class"]

# Step 3: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

```
)

# ----------------------------
# WITHOUT SCALING
# ----------------------------
model1 = LogisticRegression(max_iter=50, random_state=42)
model1.fit(X_train, y_train)

y_pred1 = model1.predict(X_test)
acc1 = accuracy_score(y_test, y_pred1)

# ----------------------------
# WITH MIN-MAX SCALING
# ----------------------------
scaler = MinMaxScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model2 = LogisticRegression(max_iter=50, random_state=42)
model2.fit(X_train_scaled, y_train)

y_pred2 = model2.predict(X_test_scaled)
acc2 = accuracy_score(y_test, y_pred2)

# Step 4: Compare results
print("Accuracy WITHOUT Scaling:", acc1)
print("Accuracy WITH Min-Max Scaling:", acc2)
```

Most Frequent Viva Questions (with Answers)

1. What is Min-Max scaling?

Scaling data to range between 0 and 1.

---

2. What is normalization?

Another name for Min-Max scaling.

---

3. Why use scaling?

To make all features equal in importance.

---

4. What is the formula of Min-Max scaling?

(X - min) / (max - min)

---

5. What is MinMaxScaler?

Tool to perform normalization.

---

6. Why fit scaler on training data only?

To avoid data leakage.

---

7. What is data leakage?

Using test data information during training.

---

8. What is Logistic Regression?

Classification algorithm.

---

9. Why scaling helps Logistic Regression?

Improves convergence and performance.

---

10. What is accuracy?

Correct predictions percentage.

---

11. What happens without scaling?

Large values dominate the model.

---

12. What is difference between Min-Max and Standardization?

- Min-Max → 0 to 1
- Standardization → mean 0, std 1

---

13. Which algorithms need scaling?

KNN, SVM, Logistic Regression.

---

14. Which do not need scaling?

Decision Trees, Random Forest.

15. What is advantage of Min-Max scaling?

Simple and bounded values.

---

16. What is disadvantage of Min-Max scaling?

Sensitive to outliers.

---

17. What is overfitting?

Model performs well on training but not test data.

---

18. What is underfitting?

Model fails to capture patterns.

---

19. Why transform test data separately?

To apply same scaling as training.

---

20. What is main benefit of scaling?

Better and faster model performance.

10. Evaluate the performance of a classification model using confusion matrix, accuracy, precision, recall, and F1-score.

(Dataset: Iris.csv)

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

# Step 1: Load dataset
```

```
df = pd.read_csv("Iris.csv")

# Step 2: Separate features and target
X = df.drop("species", axis=1)
y = df["species"]

# Step 3: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 4: Train model
model = LogisticRegression(max_iter=200, random_state=42)
model.fit(X_train, y_train)

# Step 5: Prediction
y_pred = model.predict(X_test)

# Step 6: Evaluation metrics
cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred, average="macro")
recall = recall_score(y_test, y_pred, average="macro")
f1 = f1_score(y_test, y_pred, average="macro")

# Step 7: Print results
print("Confusion Matrix:\n", cm)
print("\nAccuracy:", acc)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

Most Frequent Viva Questions (with Answers)

1. What is confusion matrix?

A table showing actual vs predicted values.

---

2. What are TP, TN, FP, FN?

- TP: correct positive
- TN: correct negative
- FP: wrong positive
- FN: wrong negative

3. What is accuracy?

Correct predictions divided by total predictions.

---

4. What is precision?

Correct positive predictions out of predicted positives.

---

5. What is recall?

Correct positive predictions out of actual positives.

---

6. What is F1 score?

Harmonic mean of precision and recall.

---

7. Why use multiple metrics?

To better evaluate model performance.

---

8. What is macro average?

Average metric across all classes equally.

---

9. What is classification report?

Summary of precision, recall, F1-score.

---

10. What is Logistic Regression?

Classification algorithm.

---

11. What is multi-class classification?

More than two output classes.

---

12. Why accuracy is not enough?

It may mislead in imbalanced data.

13. What is imbalanced data?

Unequal number of samples in classes.

---

14. What is TP?

Correctly predicted positive.

---

15. What is FP?

Incorrectly predicted positive.

---

16. What is FN?

Missed actual positive.

---

17. What is TN?

Correctly predicted negative.

---

18. What does predict() do?

Generates predictions.

---

19. What does fit() do?

Trains the model.

---

20. What is main goal of evaluation?

To measure how well model performs.

11. Evaluate the performance of a classification model using confusion matrix, accuracy, precision, recall, and F1-score.

(Dataset: diabetes.csv)

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

# Step 1: Load dataset
df = pd.read_csv("diabetes.csv")

# Step 2: Separate features and target
X = df.drop("Outcome", axis=1)
y = df["Outcome"]

# Step 3: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 4: Train model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Step 5: Prediction
y_pred = model.predict(X_test)

# Step 6: Evaluation
cm = confusion_matrix(y_test, y_pred)
acc = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

# Step 7: Print results
print("Confusion Matrix:\n", cm)
print("Accuracy:", acc)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

Most Frequent Viva Questions (with Answers)

1. What is the goal of this program?

To predict whether a person has diabetes.

---

2. What type of problem is this?

Binary classification.

3. What is Random Forest?

An ensemble algorithm using multiple decision trees.

---

4. What is n_estimators?

Number of trees in the forest.

---

5. What is confusion matrix?

Table showing actual vs predicted values.

---

6. What are TP, TN, FP, FN?

- TP: correct positive
- TN: correct negative
- FP: wrong positive
- FN: wrong negative

---

7. What is accuracy?

Correct predictions divided by total.

---

8. What is precision?

Correct positive predictions out of predicted positives.

---

9. What is recall?

Correct positive predictions out of actual positives.

---

10. What is F1 score?

Harmonic mean of precision and recall.

---

11. Why Random Forest is better than single tree?

Reduces overfitting and improves accuracy.

12. What is overfitting?

Model performs well on training but poorly on test data.

---

13. What is underfitting?

Model fails to capture patterns.

---

14. What does fit() do?

Trains the model.

---

15. What does predict() do?

Predicts output.

---

16. Why use multiple metrics?

To get complete evaluation.

---

17. What is binary classification?

Two output classes (0 or 1).

---

18. What is ensemble learning?

Combining multiple models.

---

19. What is majority voting?

Final prediction based on most common result.

---

20. What is main advantage of Random Forest?

High accuracy and robustness.

12. Evaluate clustering results using Silhouette Score and Davies-Bouldin Index. (Dataset: Mall_Customers.csv)

```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Step 1: Load dataset
df = pd.read_csv("Mall_Customers.csv")

# Step 2: Drop unnecessary columns
df = df.drop(["CustomerID", "Gender"], axis=1)

# Step 3: Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# Step 4: Apply KMeans clustering
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)

# Step 5: Visualize clusters (first two features)
plt.figure(figsize=(8, 6))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap="viridis")
plt.title("K-Means Clusters")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()

# Step 6: Evaluate clustering
S = silhouette_score(X_scaled, labels)
DB = davies_bouldin_score(X_scaled, labels)

print("Silhouette Score:", S)
print("Davies-Bouldin Index:", DB)
```

Most Frequent Viva Questions (with Answers)

1. What is clustering?

Grouping similar data points.

---

2. What is KMeans?

Algorithm that groups data based on distance.

---

3. Why scaling is required?

To ensure equal importance of features.

4. What is Silhouette Score?

Measure of how well data fits in its cluster.

---

5. What is range of Silhouette Score?

-1 to 1.

---

6. What does high Silhouette Score mean?

Better clustering.

---

7. What is Davies-Bouldin Index?

Measure of cluster similarity.

---

8. What is range of DB Index?

0 to infinity.

---

9. What does low DB Index mean?

Better clustering.

---

10. What is centroid?

Center of cluster.

---

11. What is label in clustering?

Cluster number assigned to data point.

---

12. What is fit_predict()?

Fits model and returns cluster labels.

---

13. Why remove CustomerID?

It has no meaningful information.

---

14. Why remove Gender?

Because it is categorical (not numeric here).

---

15. What is unsupervised learning?

Learning without labeled data.

---

16. What is distance in KMeans?

Used to assign clusters.

---

17. What is over-clustering?

Too many clusters.

---

18. What is under-clustering?

Too few clusters.

---

19. Why visualize clusters?

To understand grouping.

---

20. What is main goal of evaluation?

To check clustering quality.

13. Evaluate clustering results using Silhouette Score and Davies-Bouldin Index. (Dataset: diabetes.csv)

```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Step 1: Load dataset
df = pd.read_csv("diabetes.csv")
```

```
# Step 2: Select features
X = df[["Glucose", "BMI"]]

# Step 3: Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 4: Apply KMeans clustering
kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)

# Step 5: Plot clusters
plt.figure(figsize=(8, 6))
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap="viridis")
plt.xlabel("Glucose")
plt.ylabel("BMI")
plt.title("K-Means Clustering")
plt.colorbar(label="Cluster")
plt.show()

# Step 6: Evaluate clustering
S = silhouette_score(X_scaled, labels)
DB = davies_bouldin_score(X_scaled, labels)

print("Silhouette Score:", S)
print("Davies-Bouldin Index:", DB)
```

Most Frequent Viva Questions (with Answers)

1. What is clustering?

Grouping similar data points.

---

2. What is KMeans?

Algorithm that divides data into clusters.

---

3. Why use only two features?

For easy visualization.

---

4. Why scaling is required?

To give equal importance to features.

---

5. What is Silhouette Score?

Measure of how well points fit within clusters.

---

6. What is range of Silhouette Score?

-1 to 1.

---

7. What does high Silhouette Score mean?

Better clustering.

---

8. What is Davies-Bouldin Index?

Measure of cluster similarity.

---

9. What does low DB Index mean?

Better clustering.

---

10. What is centroid?

Center of cluster.

---

11. What is label?

Cluster number assigned to each point.

---

12. What is fit_predict()?

Fits model and returns labels.

---

13. What is unsupervised learning?

Learning without labels.

---

14. What is distance in KMeans?

Used to assign clusters.

---

15. What is over-clustering?

Too many clusters.

---

16. What is under-clustering?

Too few clusters.

---

17. Why visualize clusters?

To understand grouping.

---

18. What is advantage of KMeans?

Simple and fast.

---

19. What is limitation of KMeans?

Sensitive to outliers.

---

20. What is goal of evaluation?

To check clustering quality.

14. Implement a Random Forest classifier and compare its performance with a Decision Tree classifier.

(Dataset: Titanic.csv)

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Step 1: Load dataset
df = pd.read_csv("Titanic.csv")

# Step 2: Select relevant features
df = df[["Survived", "Pclass", "Sex", "Age", "Fare"]]

# Step 3: Handle missing values
df["Age"] = df["Age"].fillna(df["Age"].median())
```

```
# Step 4: Encode categorical data
le = LabelEncoder()
df["Sex"] = le.fit_transform(df["Sex"])

# Step 5: Separate features and target
X = df.drop("Survived", axis=1)
y = df["Survived"]

# Step 6: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------
# Decision Tree Model
# ----------------------------
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

# ----------------------------
# Random Forest Model
# ----------------------------
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

# Step 7: Compare performance
dt_acc = accuracy_score(y_test, dt_pred)
rf_acc = accuracy_score(y_test, rf_pred)

print("Decision Tree Accuracy:", dt_acc)
print("Random Forest Accuracy:", rf_acc)
```

Most Frequent Viva Questions (with Answers)

1. What is Decision Tree?

A model that splits data based on conditions.

---

2. What is Random Forest?

An ensemble of multiple decision trees.

---

3. What is ensemble learning?

Combining multiple models.

---

4. Why Random Forest is better?

Reduces overfitting and increases accuracy.

---

5. What is n_estimators?

Number of trees in Random Forest.

---

6. What is overfitting?

Model fits training data too well but fails on test data.

---

7. What is underfitting?

Model fails to capture patterns.

---

8. What is accuracy?

Correct predictions divided by total.

---

9. Why encode categorical data?

Models require numeric input.

---

10. Why fill missing Age?

Model cannot handle missing values.

---

11. Why use median instead of mean?

Median is less affected by outliers.

---

12. What is splitting in Decision Tree?

Dividing data based on feature conditions.

---

13. What is leaf node?

Final output node in tree.

---

14. What is root node?

Top node of tree.

---

15. What is majority voting?

Final decision based on most common prediction.

---

16. What is random in Random Forest?

Random selection of data and features.

---

17. Which model is faster?

Decision Tree (simpler).

---

18. Which model is more accurate?

Random Forest (generally).

---

19. What is bagging?

Training multiple models on different subsets.

---

20. What is main advantage of Random Forest?

High accuracy and robustnes

15. Implement an XGBoost classifier and compare its performance with a Decision Tree classifier.

(Dataset: Titanic.csv)

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from xgboost import XGBClassifier

# Step 1: Load dataset
df = pd.read_csv("Titanic.csv")
```

```
# Step 2: Select relevant features
df = df[["Survived", "Pclass", "Sex", "Age", "Fare"]]

# Step 3: Handle missing values
df["Age"] = df["Age"].fillna(df["Age"].median())

# Step 4: Encode categorical data
le = LabelEncoder()
df["Sex"] = le.fit_transform(df["Sex"])

# Step 5: Separate features and target
X = df.drop("Survived", axis=1)
y = df["Survived"]

# Step 6: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ------------------------------
# Decision Tree Model
# ------------------------------
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

# ------------------------------
# XGBoost Model
# ------------------------------
xgb = XGBClassifier(
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)
xgb.fit(X_train, y_train)
xgb_pred = xgb.predict(X_test)

# Step 7: Compare performance
dt_acc = accuracy_score(y_test, dt_pred)
xgb_acc = accuracy_score(y_test, xgb_pred)

print("Decision Tree Accuracy:", dt_acc)
print("XGBoost Accuracy:", xgb_acc)
```

Most Frequent Viva Questions (with Answers)

1. What is Decision Tree?

A model that splits data using conditions.

2. What is XGBoost?

An advanced boosting algorithm using multiple trees.

---

3. What is boosting?

Training models sequentially to correct errors.

---

4. What is difference between bagging and boosting?

- Bagging → parallel models
- Boosting → sequential models

---

5. Why XGBoost performs better?

It reduces errors step by step.

---

6. What is eval_metric='logloss'?

Loss function used for classification.

---

7. What is accuracy?

Correct predictions divided by total.

---

8. What is overfitting?

Model fits training data too well.

---

9. How XGBoost handles overfitting?

Uses regularization.

---

10. What is regularization?

Technique to reduce model complexity.

---

11. What is gradient boosting?

Method used in XGBoost.

12. What is advantage of XGBoost?

High accuracy and efficiency.

---

13. What is disadvantage of XGBoost?

More complex and slower.

---

14. Why encode categorical data?

Models require numeric input.

---

15. Why fill missing values?

Model cannot handle missing data.

---

16. What is random_state?

Ensures reproducible results.

---

17. Which model is simpler?

Decision Tree.

---

18. Which model is more powerful?

XGBoost.

---

19. What is ensemble learning?

Combining multiple models.

---

20. What is main goal of comparison?

To evaluate which model performs better.

16. Implement a Random Forest classifier and compare its performance with a Decision Tree classifier.

(Dataset: Wine.csv)

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Step 1: Load dataset
df = pd.read_csv("wine.csv")

# Step 2: Separate features and target
X = df.drop("Class", axis=1)
y = df["Class"]

# Step 3: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ------------------------------
# Decision Tree Model
# ------------------------------
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

# ------------------------------
# Random Forest Model
# ------------------------------
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

# Step 4: Compare performance
dt_acc = accuracy_score(y_test, dt_pred)
rf_acc = accuracy_score(y_test, rf_pred)

print("Decision Tree Accuracy:", dt_acc)
print("Random Forest Accuracy:", rf_acc)
```

Most Frequent Viva Questions (with Answers)

1. What is Decision Tree?

A model that splits data using conditions.

---

2. What is Random Forest?

Collection of multiple decision trees.

3. What is ensemble learning?

Combining multiple models.

4. What is n_estimators?

Number of trees in Random Forest.

5. Why Random Forest is better?

Reduces overfitting and improves accuracy.

6. What is overfitting?

Model performs well on training but poorly on test.

7. What is underfitting?

Model fails to learn patterns.

8. What is accuracy?

Correct predictions divided by total.

9. What is majority voting?

Final prediction based on most common result.

10. What is bagging?

Training models on random subsets.

11. Which model is faster?

Decision Tree.

12. Which model is more accurate?

Random Forest (generally).

13. What is root node?

Top node of tree.

---

14. What is leaf node?

Final decision node.

---

15. What is splitting?

Dividing data based on conditions.

---

16. What is feature importance?

Measure of how useful a feature is.

---

17. What type of problem is this?

Classification.

---

18. What is supervised learning?

Learning using labeled data.

---

19. What is main advantage of Random Forest?

High accuracy and stability.

---

20. What is limitation of Decision Tree?

Prone to overfitting.

17. Implement an XGBoost classifier and compare its performance with a Decision Tree classifier.

(Dataset: Titanic.csv)

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from xgboost import XGBClassifier

# Step 1: Load dataset
df = pd.read_csv("wine.csv")

# Step 2: Separate features and target
X = df.drop("Class", axis=1)
y = df["Class"]

# Step 3: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------
# Decision Tree Model
# ----------------------------
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

# ----------------------------
# XGBoost Model
# ----------------------------
xgb = XGBClassifier(eval_metric='mlogloss', random_state=42)
xgb.fit(X_train, y_train)
xgb_pred = xgb.predict(X_test)

# Step 4: Compare performance
dt_acc = accuracy_score(y_test, dt_pred)
xgb_acc = accuracy_score(y_test, xgb_pred)

print("Decision Tree Accuracy:", dt_acc)
print("XGBoost Accuracy:", xgb_acc)
```

Most Frequent Viva Questions (with Answers)

1. What is Decision Tree?

A model that splits data based on conditions.

---

2. What is XGBoost?

An advanced boosting algorithm.

---

3. What is boosting?

Sequential learning to reduce errors.

---

4. What is difference between bagging and boosting?

- Bagging → parallel
- Boosting → sequential

---

5. Why XGBoost performs better?

It corrects previous errors.

---

6. What is eval_metric='mlogloss'?

Loss function for multi-class classification.

---

7. What is accuracy?

Correct predictions divided by total.

---

8. What is overfitting?

Model performs well on training but poorly on test.

---

9. How XGBoost reduces overfitting?

Using regularization.

---

10. What is regularization?

Reducing model complexity.

---

11. What is gradient boosting?

Technique used in XGBoost.

---

12. What is advantage of XGBoost?

High accuracy and efficiency.

---

13. What is disadvantage of XGBoost?

Complex and slower.

---

14. What type of problem is this?

Classification.

---

15. What is supervised learning?

Learning using labeled data.

---

16. Which model is simpler?

Decision Tree.

---

17. Which model is more powerful?

XGBoost.

---

18. What is ensemble learning?

Combining multiple models.

---

19. What is main goal of comparison?

To evaluate performance.

---

20. What is key strength of XGBoost?

Better accuracy through boosting.

18. Apply Recursive Feature Elimination (RFE) to select the most relevant features for model building.

(Dataset: Titanic.csv)

```
# Import required libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.feature_selection import RFE
```

```
# Step 1: Load dataset
df = pd.read_csv("Titanic.csv")

# Step 2: Select relevant features
df = df[["Survived", "Pclass", "Sex", "Age", "Fare"]]

# Step 3: Handle missing values
df["Age"] = df["Age"].fillna(df["Age"].median())

# Step 4: Encode categorical data
le = LabelEncoder()
df["Sex"] = le.fit_transform(df["Sex"])

# Step 5: Separate features and target
X = df.drop("Survived", axis=1)
y = df["Survived"]

# Step 6: Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 7: Create model
model = LogisticRegression(max_iter=1000)

# Step 8: Apply RFE (select top 2 features)
rfe = RFE(estimator=model, n_features_to_select=2)
rfe.fit(X_train, y_train)

# Step 9: Get selected features
selected_features = X.columns[rfe.support_]

print("Selected Features:", list(selected_features))
```

Most Frequent Viva Questions (with Answers)

1. What is RFE?

A feature selection method that removes less important features.

---

2. What does RFE stand for?

Recursive Feature Elimination.

---

3. How does RFE work?

Removes weakest features step-by-step.

---

4. Why use RFE?

To select most important features.

---

5. What is feature selection?

Choosing important input variables.

---

6. What is estimator in RFE?

Model used to evaluate feature importance.

---

7. Why Logistic Regression is used here?

It provides feature importance through coefficients.

---

8. What is n_features_to_select?

Number of features to keep.

---

9. What is support_ in RFE?

Boolean array showing selected features.

---

10. What is ranking_ in RFE?

Ranks features based on importance.

---

11. What is advantage of RFE?

Improves model performance.

---

12. What is disadvantage of RFE?

Computationally expensive.

---

13. What type of method is RFE?

Wrapper method.

---

14. What is wrapper method?

Uses model performance to select features.

---

15. What is filter method?

Selects features based on statistics.

---

16. What is embedded method?

Feature selection during model training.

---

17. Why reduce features?

To reduce overfitting and complexity.

---

18. What is overfitting?

Model fits training data too well.

---

19. What is underfitting?

Model fails to learn patterns.

---

20. What is main goal of RFE?

Select best features for model building.